CHALLENGE SHEET

# Firmdrama

A law firm's meeting-room portal with chained API authorization flaws and a vulnerable legacy template renderer.

Created by Ziad Osama El-Boshy

| DIFFICULTY | TRACK | HOST PORT | LAB ID |
| --- | --- | --- | --- |
| HARD | WEB | 8080 | firmdrama |

EASY · MEDIUM · HARD · INSANE

VERIFY THE SOLVE

```
python3 ../scripts/check.py firmdrama
```

## Brief

RoomReserve coordinates confidential meetings, room bookings, and routine Facilities requests at the fictional Alder & Vale law firm. You are Mike, an associate with an ordinary firm account. A partner booking has caught your attention, and the application may reveal more than it should.

## Goal

Investigate the application's observable REST API behavior and retrieve the challenge flags. The browser interface supports everyday RoomReserve tasks; the challenge can also be completed entirely with an HTTP client.

> **Your starting point:** an ordinary employee account, a working portal, and the information the application makes available to you.

## Prerequisites

- Docker Desktop using Linux containers, or Docker Engine on Linux, with Docker Compose v2 or later.
- Python 3.10 or later on your host for the one-time hash synchronizer, reset helper, and local checker; no additional Python packages are required.
- A browser with developer tools, or an HTTP client such as curl, Postman, or Burp Suite.
- Basic familiarity with HTTP methods, JSON, and bearer-token authentication.
- A Docker Engine that supports isolated bridge gateway mode (validated with Engine 29.6.2). Unsupported engines must be upgraded; keep the isolation option.
- An available local port 8080. The first build requires internet access to download images and packages; the running lab needs no external service.

# Setup

## 1. Start RoomReserve

Open a terminal in the `firmdrama` directory and run:

```
docker compose up -d --build --wait
docker compose ps
```

The first build may take several minutes. Wait until **both services show `healthy`** before continuing.

## 2. Initialize the flag hashes

Once both services are healthy, run this once in the same terminal:

```
python scripts/sync_flag_hash.py
```

If your system uses `python3`, substitute it for `python`. The command updates both SHA-256 hashes in lab.yml, or confirms that they already match, and exits. It never prints the flags. If it fails, check that the services are healthy and rerun it before playing.

Repeat this command after starting or recreating the lab with Docker directly. `python reset.py` updates both hashes itself; no separate sync is needed after that command succeeds.

## 3. Open the portal and sign in

Open **RoomReserve → http://127.0.0.1:8080** and use the supplied challenge account:

| Connection detail | Value |
|---|---|
| Username | `thisismike` |
| Password | `mike@123` |
| Readiness check | GET /health |

The health response is:

```
\{"service":"firmdrama","status":"ok"\}
```

**If the page does not open**

Confirm Docker is running with Linux containers, then check `docker compose ps`. Both services must be healthy. If startup fails, inspect the recent logs:

```
docker compose logs --tail 100
```

If port `8080` is already occupied, stop the conflicting local service and retry. Keep the supplied loopback binding for local use.

## 4. Begin your investigation

Explore the portal and its API, follow the evidence, and collect the challenge flags. Each flag uses this format:

| Flag requirement | Value |
| --- | --- |
| **Format** | `duck{` + exactly 24 lowercase letters (`a-z`) + `}` |
| **Total length** | 30 characters |
| **Number of flags** | 2 |

## Check a captured flag

After collecting either flag, verify it locally from the `firmdrama` directory with the shared repository checker:

```
python ../../../scripts/check.py .
```

Paste one captured value at the prompt. The checker reports `checkpoint solved` for the intermediate flag and `solved` for the final flag; any other value returns `not solved`. It compares a SHA-256 digest only, never sends the flag anywhere, and never prints a stored flag.

The checker uses the current hashes in lab.yml. Initialize them once after Docker startup as described above. `reset.py` refreshes them automatically. After a direct Docker restart or recreation, rerun `python scripts/sync_flag_hash.py` before checking a flag. In hosted delivery, the checker needs the matching instance's `lab.yml`.

## Reset the lab

Reset restores the starting records and account state, invalidates active sessions, and generates fresh flags. Current progress is discarded; sign in again afterward.

```
python reset.py
```

Run this on the host from the firmdrama directory. It recreates both containers from existing images, clears all progress and temporary processes/files, verifies fresh state, and refreshes both checker hashes. It does not rebuild or download images. Sign in again after the success message. If reset fails, ingress stays stopped; resolve the reported error and rerun the command.

`reset.py` refreshes both hashes before reporting success.

## Stop the lab

```
docker compose down
```

The lab uses temporary storage. Stopping its containers discards the in-memory data; the next startup initializes a fresh challenge state. Initialize the hashes again after starting the next session.

## Player rules

- Work only within this authorized, isolated challenge environment.

- Do not target public systems, third-party services, or real accounts.
- Do not use persistence, destructive actions, or external callbacks.
- Keep challenge data and captured flags inside the authorized lab.
- In hosted delivery, only the assigned HTTP(S) endpoint is in scope. Host, Docker, database, image, source, solver, and reset controls belong to the operator and are not player-accessible assets.

---

**firmdrama · Web/API** A fictional workplace. A hands-on API investigation.